PROJECT REPORT

MBA 666

Low Code or No Code

The new era of Project Management

Submitted by:

Diya Saraf(220383)

Harshal Dharme(241250023)

Kavita(220512)

Sharanbasappa K(241250048)

Shrimi Agrawal(221036)

1. Introduction

1.1 Background:

Traditional software development is based heavily on manual coding; developers write every line of code from scratch. This approach offers complete control and customisation, but it requires deep technical expertise, large teams, time, and investments.

This limitation led to the emergence of **Low-Code (LC)** and **No-Code (NC)** platforms, which simplify software creation through **visual interfaces**, **drag-and-drop components**, and **model-driven design**. Examples include **Microsoft Power Apps**, **Appian**, **Mendix**, **Quickbase**, and **Airtable**.Low-Code platforms still allow flexibility for complex enterprise-grade applications, while No-Code tools focus on simplicity and speed for non-technical users.

1.2 Problem Statement/Research Gap:

This platform are growing but organizations often struggle to understand how Low-Code and No-Code platforms reshape **project management** practices. And still there is limited research on their **strategic impact on project management methodologies**, particularly on **governance, risk control, team dynamics, and the role of the project manager**.

This creates a clear research gap: how do LC/NC platforms influence the traditional project management constraints of **time, cost, and quality**, and what new challenges arise from their adoption, such as **shadow IT**, **governance complexity**, and **citizen development**?

1.3 Research Objectives:

The aim of this project is to explore how Low-Code and No-Code tools are reshaping modern project management.

- To explain the principles and evolution of Low-Code and No-Code platforms.
- To analyze how LC/NC tools accelerate project delivery and improve flexibility across different industries.
- To examine their impact on project management practices, particularly in relation to Agile methodologies and governance.
- To compare LC/NC approaches with traditional development in terms of time, cost, and quality.

- To identify practical lessons and recommendations for project managers adopting LC/NC in real-world scenarios.

1.4 Scope of the Study:

The scope includes analysis of **industry case studies** such as **Schneider Electric (Quickbase)**, **Pfizer (Appian)**, and **G&J Pepsi (Microsoft Power Platform)** to highlight real-world applications.

The study does not aim to provide technical coding details, but instead focuses on **strategic and managerial implications** — how project managers can effectively leverage LC/NC platforms to enhance delivery speed, maintain governance, and achieve sustainable business outcomes.

2. Literature Review

2.1 Defining Low Code and No Code

Low-Code/No-Code (LCNC) platforms democratize the ability to create applications and automate processes, allowing Project Managers (PMs) to rapidly address business needs that previously stalled in the IT backlog [17]. The modern definition of LCNC extends far beyond traditional software projects, encompassing the rapid development of **dashboards, websites, automated reports, and digital workflows** for every department. LCNC is based on abstracting traditional code through visual interfaces and model-driven design [20].

2.1.1 Distinguishing Low-Code (LC) from No-Code (NC)

The distinction between the two approaches primarily concerns the complexity of the solution and the required user skill set [18]:

Feature	Low Code Platform(LC)	No Code Platform (NC)
Target User	Professional Developers, Citizen Developers with basic coding skills, and tech-savvy Project Managers [18]	Business Users (Citizen Developers) with **zero coding experience** [17]
Complexity	Suitable for building **complex, enterprise-grade applications** [17]	Generally limited to simple, **tactical apps** (forms, dashboards, simple workflows) [17]
Flexibility	**Open system** that allows for advanced customization and script	**Closed system** that relies 100% on templated, plug-and-play

	extensions over auto-generated code [18]	features, restricting customization [18]

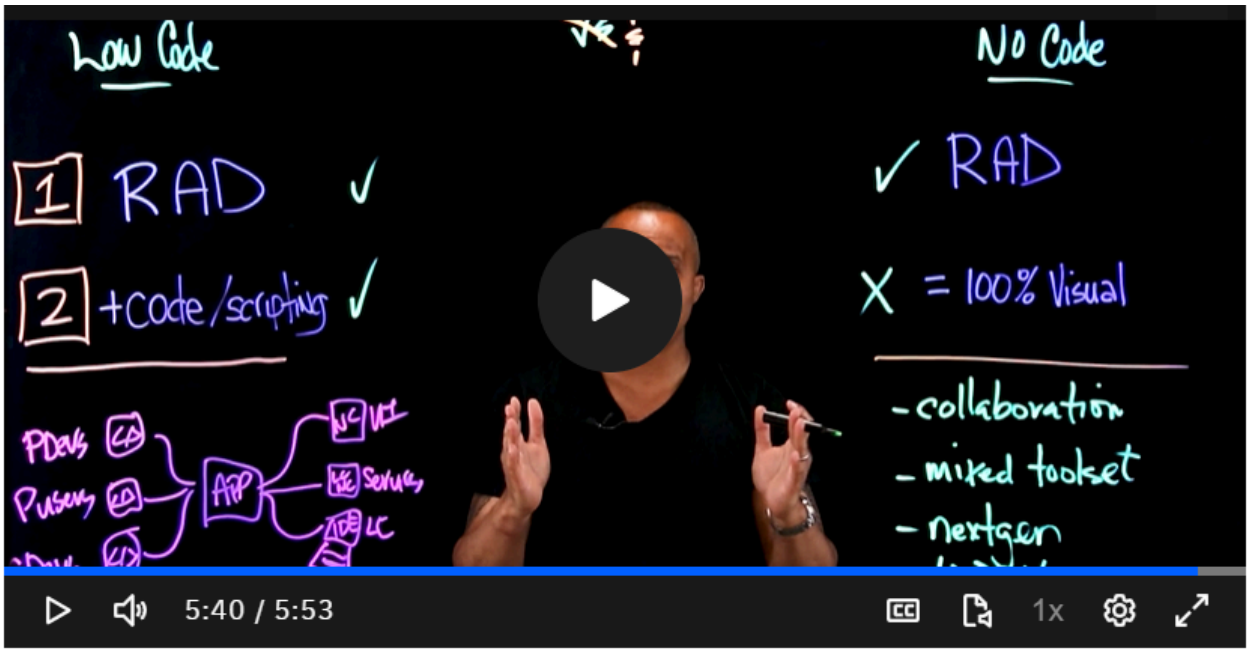


Fig:Diagram illustrating the **distinction between Low-Code and No-Code** [1]

2.1.2 Core Shift from Pro-Code Development

The move to LCNC fundamentally streamlines the traditional **Software Development Lifecycle (SDLC)** by minimizing manual coding and shifting focus from syntax to business outcomes [20].

- **Visual, Model-Driven Design:** LCNC platforms use intuitive graphical interfaces, drag-and-drop features, and pre-built components to **automate every step** of the development process [19, 20]. This makes development significantly **faster**—in some cases, up to 20 times faster than traditional coding [21].
- **Empowering the Citizen Developer:** LCNC empowers **Citizen Developers** across the business to create their own solutions, reducing reliance on the central IT department [17]. This democratization spurs innovation and dramatically **reduces the strain and backlogs** on specialized development teams [19].
- **Focus on Process Orchestration:** Contrary to the belief that it is only for apps, LCNC's real value lies in its role as a **strategic instrument for workflow intelligence and enterprise transformation** (User Context). This allows PMs to

focus projects on automating business processes (like procurement or compliance) rather than custom application development [17].

2.2 The Evolution of Visual Development:Tracing the Path to LCNC

The **Iron Triangle** defines project success by the balance between **Scope, Time, and Cost**, with **Quality** traditionally existing as the result of the trade-offs between them [22]. LCNC platforms are transforming this model by enabling simultaneous gains across all constraints.

2.2.1 Horizon I: Abstraction through Declarative Languages (4GLs)

TThe first step away from traditional, line-by-line procedural coding (3GLs like COBOL and C) was the introduction of **Fourth-Generation Languages (4GLs)** in the 1970s and 1980s. [23]

- **Principle:** 4GLs were **non-procedural**, meaning the user specified *what* needed to be done (the desired result) rather than explicitly coding *how* to achieve it (the step-by-step logic).
- **Focus:** They were domain-specific, primarily designed for tasks like:
    - **Database query** (e.g., SQL)
    - **Report generation**
    - **Screen definition** (early form builders)
- **Impact on PM:** This provided the first tool to dramatically increase **developer productivity** for common business tasks, laying the groundwork for the future concept of high-velocity development.

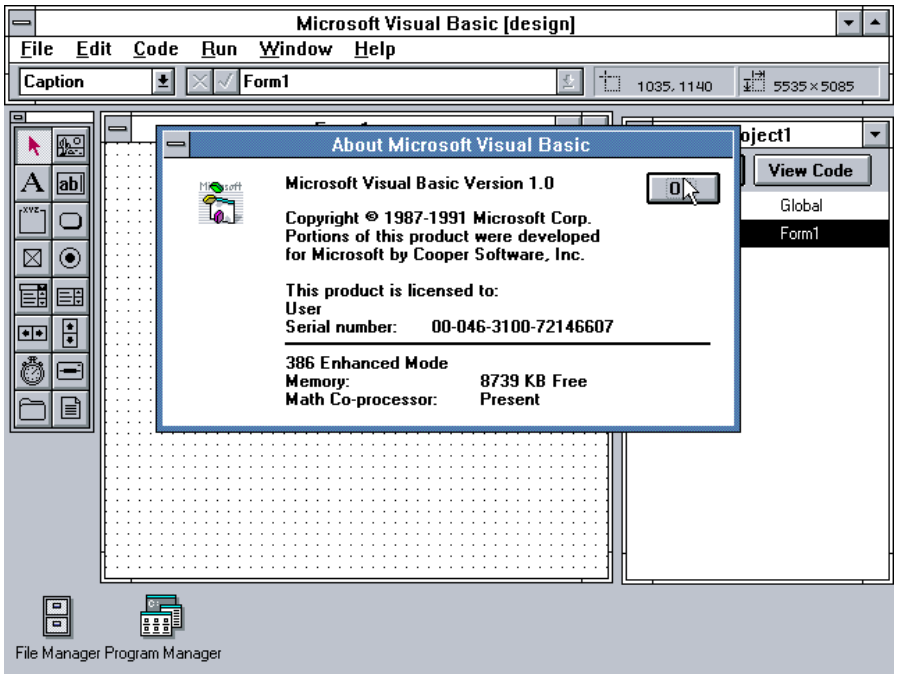


Fig:Screenshot of early Microsoft Visual Basic interface[22]

2.2.2 Horizon II: Visualization and Iteration (RAD)

The 1990s formalized the use of visual tools with the **Rapid Application Development (RAD)** methodology, primarily associated with James Martin.

- **Tools:** Platforms like **Visual Basic** and **Delphi** provided the first true **Graphical User Interface (GUI) builders** with drag-and-drop components, enabling developers to visually assemble forms and simple applications. [24]
- **Methodology:** The core of RAD centered on:
  - **Iterative Development:** Building the solution in short, repeating cycles.
  - **Prototyping:** Producing functional, visible models quickly for continuous user feedback.
- **Impact on PM:** This generation emphasized the constraint of **Time**. The ability to produce a **functional prototype** rapidly became a critical project milestone, providing immediate stakeholder visibility and accelerating the crucial feedback loop

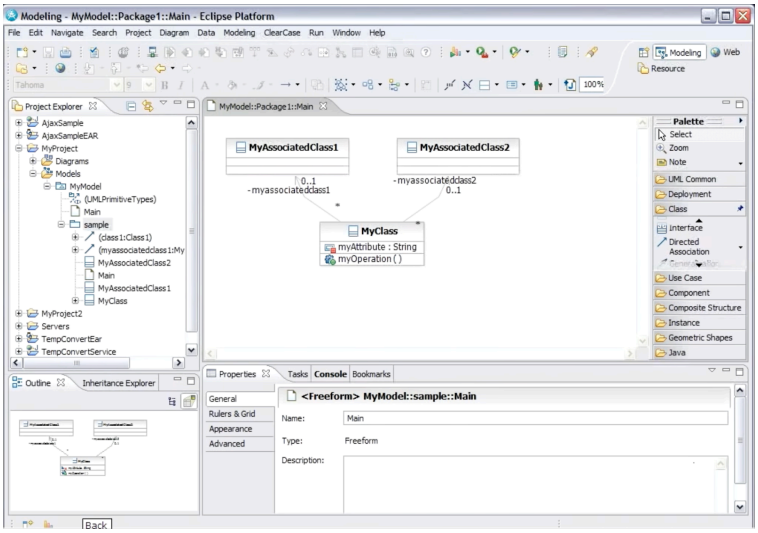


Fig: Screenshot of early IBM Rational Software Modeler interface[22]

2.2.3 Horizon III: Convergence and Orchestration (LCNC)

Modern Low-Code/No-Code (LCNC) platforms represent the synthesis of 4GL abstraction and RAD visualization, further enhanced by cloud technology, enterprise integration, and robust governance. [23]

- **Core Advancement:** LCNC moves beyond building isolated applications (the focus of RAD) to become **platforms for process orchestration**. They integrate multiple systems, manage data flows, and automate end-to-end business journeys (e.g., procurement, compliance).
- **New Components:** Built-in features ensure that speed is achieved without compromising enterprise quality or introducing Shadow IT risks:
    - Pre-configured **connectors** for deep system integration.
    - Centralized **governance models (CoE frameworks)** for Citizen Development.
    - Cloud-native architecture for scalability and security.
- **Impact on PM:** LCNC transforms all four constraints (Scope, Time, Cost, and **Quality**) simultaneously, enabling PMs to manage solutions that are highly agile, deeply integrated, and scalable across the organization.

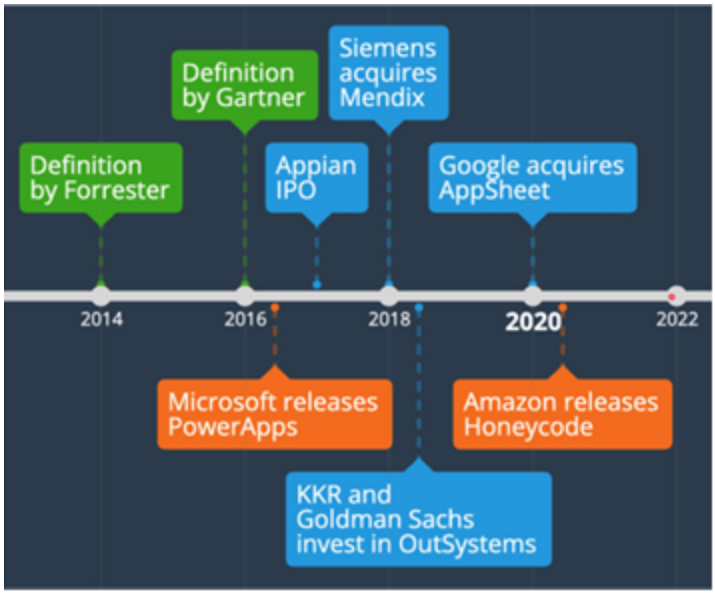


Fig:Major Events in low-code history[23]

2.3 LC/NC in the Context of Project Management

This section is about the paradigm shift about how projects are run and managed when new tools are introduced(LC/NC tools)

2.3.1 Acceleration of Methodologies and Cross-Industry Flow

LC/NC is an enabler of speed because it visually abstracts complexity, fundamentally supporting the core principles of rapid delivery, whether the project involves code or concrete.

- **Impact on Agile Flow:** The essence of LC/NC—visual, model-driven development—accelerates the delivery of a **Minimum Viable Product (MVP)** [1]. This reduces the ambiguity that frequently stalls projects, as the Project Manager (PM) can facilitate stakeholder reviews of a working solution (a digitized form, a workflow, a prototype app) far faster than traditional methods allow [2]. This capability allows project teams to achieve genuine **Agile iterations** and a rapid **time-to-market** for essential business processes across all sectors [3].
- **Built-in Process Automation:**For non-software projects (e.g., construction), the PM utilizes LC/NC to move from cumbersome paper-based processes (like safety audits or material requisitions) to instant digital workflows. Many high-quality platforms automate the entire **DevOps pipeline** or the **Application Lifecycle Management (ALM)** process, which shifts the PM's

focus from coordinating manual releases to managing the platform's automated compliance and governance rules [4].

2.3.2 The Citizen Developer and Organizational Governance

The emergence of the Citizen Developer—a business expert who creates solutions using LC/NC tools [5]—is the most profound organizational change, creating both a massive resource boost and a critical governance challenge for the PM.

- **Empowerment and Shadow IT Risk:**Citizen development quickly addresses the long-standing IT Backlog [6] by empowering domain experts to build needed solutions. However, if unmanaged, this decentralization leads to **Shadow IT** because of ungoverned applications [7], introducing critical project risks such as data inconsistency, security vulnerabilities, and non-compliance [8]
- **The PM's Governance Role:** The PM must proactively establish a **Center of Excellence (CoE)** framework [9]. This framework serves as the centralized authority, providing guardrails (templates, security policies, and approved data sources) to ensure speed does not compromise enterprise stability [6, 10]. The PM's responsibility is to be the champion of this CoE, managing the platform's compliance and access control to mitigate the risk of accumulated technical debt caused by chaotic, unmaintainable solutions [8].

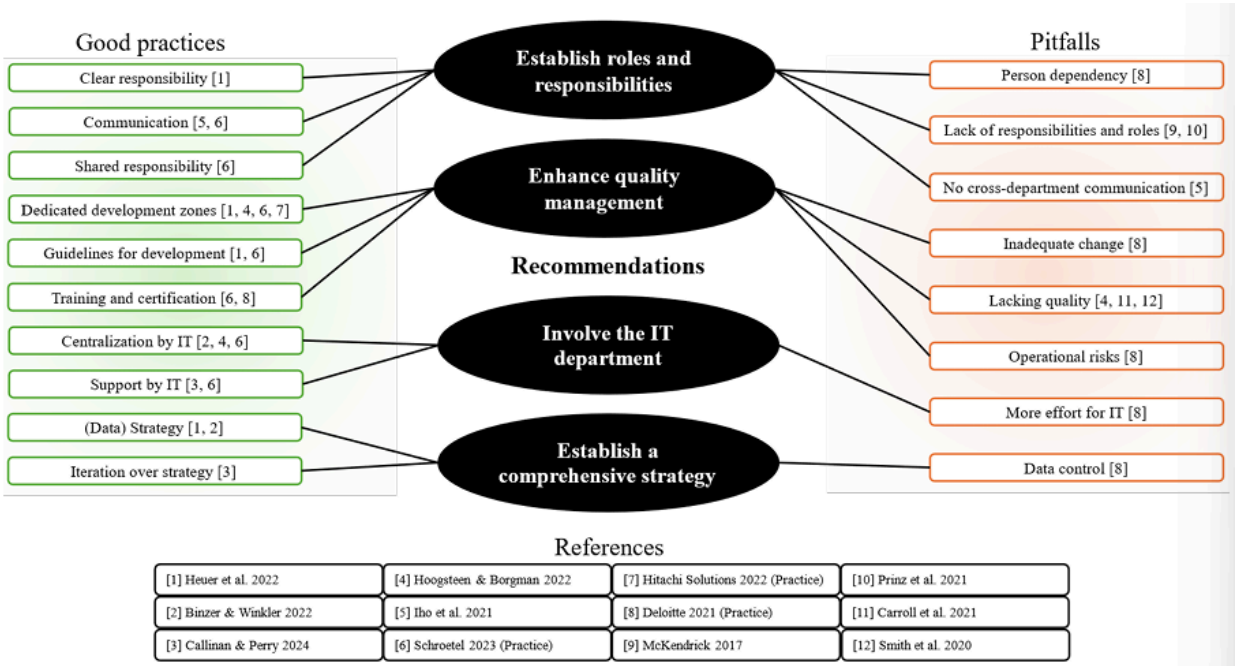


Fig. 1: Derived recommendations for NLCP and CD governance.

Fig: This diagram illustrates the four core recommendations for establishing **No-Code/Low-Code Platform (NLCP)** and **Citizen Development (CD) governance**. It visually connects Good practices (like training and clear responsibility) to these recommendations, which are strategically designed to prevent organizational Pitfalls (e.g., operational risks and lack of quality).[9]

2.4 Comparison to Traditional Development

The decision to use LC/NC is a **strategic project choice** involving a critical trade-off analysis that directly impacts project constraints (Time, Cost, Quality) for any type of project.

2.4.1 Strategic Trade-Off: Customization vs. Project Velocity

The difference is a fundamental choice between maximum control and maximum speed.

- **Traditional (Pro-Code) Control:** Traditional, manual, or pro-code development is essential when a project requires **unlimited customization** and absolute control over the core architecture (e.g., highly specialized calculations, unique legacy system integrations, or complex regulatory compliance) [11]. The PM must accept a compromise on **Project Velocity** due to higher resource costs and specialized labor requirements [12].
- **LC/NC Velocity and Constraint:** LC/NC delivers unmatched project velocity by leveraging reusable components and pre-built logic [13]. However, this speed requires the PM to ensure **the project scope can be constrained** to the platform's capabilities [11]. If a project requires functionality that the platform cannot handle out-of-the-box, the solution becomes unsuitable, and attempting workarounds can quickly negate the platform's intended benefits [14].

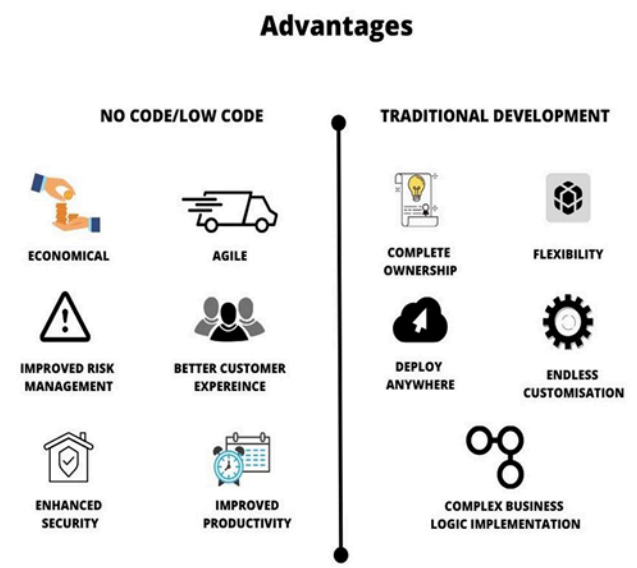


Figure 3. Advantages

Fig:This diagram summarizes the key advantages of **No Code/Low Code** (**Agile**, **Improved Productivity**) gained at the expense of speed in Traditional Development. **[12]**

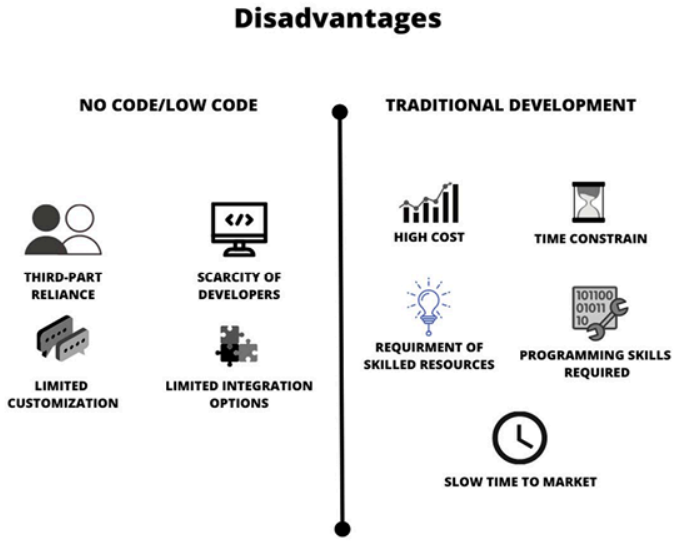


Figure 4. Disadvantages

Fig:This figure highlights the trade-off for the Project Manager by contrasting the disadvantages of both methods.[12].

2.4.2 Project Risks: Vendor Lock-in and Long-Term Viability

The Project Manager must account for the unique strategic risks associated with LC/NC platform dependency.

- **Vendor Lock-in Risk:** This is a major, long-term strategic liability [15]. LC/NC applications are built using the vendor's proprietary environment, resulting in a loss of **code ownership** [16]. This ties the project to the vendor for updates, pricing, and feature viability, creating **high switching costs** if the organization ever needs to migrate the application [15]. The PM must critically assess the platform's **exit strategy** and the **Total Cost of Ownership (TCO)** over the project's lifetime.
- **Scalability and Technical Debt:** The PM must conduct due diligence on the platform's ability to handle scalability and performance [13]. For high-volume or complex processes, the performance of the final application is governed by the underlying proprietary platform, which is a "**black box**" to the developer [16]. Furthermore, rapid development without governance can quickly lead to unmanageable **architectural debt**—an accumulation of inconsistent and hard-to-integrate project tools—which undermines the platform's initial efficiency advantage [8].

2.5 Bibliography

Ref.	URL
[1]	Low-Code vs. No-Code: What's the Difference? | IBM
[2]	What is Low-Code Development? | Mendix
[3]	Microsoft Word - 20210525_IEEE论文初稿4.doc
[4]	What is Low-Code? - ServiceNow
[5]	What is Citizen Development? Overview, Importance and Benefits 2025

[6]	Citizen Developer Governance: What it Is & Why It's Important
[7]	Is Citizen Development Shadow IT?
[8]	"Governing Citizen Development to Address Low-Code Platform Challenges" by Altus Viljoen, Marija Radić et al.
[9]	6-Step Framework for Citizen Developer Governance in 2025
[10]	(PDF) Empowering Citizen Developers: The Crucial Role of Effective Governance
[11]	Low-Code/No-Code Platforms: From Concept to Creation – IJERT
[12]	Analysis of Low Code-No Code Development Platforms in comparison with Traditional Development Methodologies
[13]	(PDF) Low-Code/No-Code Platforms and Their Impact on Traditional Software Development: A Literature Review
[14]	“No Code”, “Low Code” – Vendor Lock-In? – Digital Strategy & Insights
[15]	Vendor Lock-In Risks: Why Low-Code Platforms Must Prioritize Freedom
[16]	(PDF) Impact of Low-Code/No-Code Platforms
[17]	Reduce Your IT Backlog with Low Code for Insurance – Hitachi Solutions

[18]	What is Low-Code/No-Code Development? | Infor
[19]	Low-Code vs No-Code Platforms: Comparison, Key Differences & Use cases
[20]	Low-Code/No-Code: The Future of Development | SAP
[21]	(PDF) Low-code development and model-driven engineering: Two sides of the same coin?
[22]	The History of Low-Code
[23]	PROGRAMMING LANGUAGE GENERATIONS. – Nayeemzz Blog

3. LCNC Integration Across the Project Life Cycle

This chapter details the specific enhancements and changes Low-Code/No-Code (LCNC) platforms introduce across the five phases of the Project Management Life Cycle (PLC). This integration fundamentally shifts the PM's role from administrative oversight to strategic enablement.

3.1 Initiation Phase

The Initiation Phase defines the **strategic need, objectives, and feasibility** of the project, including preliminary analysis, business case preparation, and initial risk assessment.[5]

Traditional Goal	LCNC Enhancement to Project Initiation
Defines Strategic Need	**Rapid Prototyping and Proof of Concept (PoC):** LCNC platforms allow PMs to quickly create working prototypes without extensive development resources. This helps stakeholders **visualize the end product early** and **validate project feasibility** before committing significant capital.
Gathers Requirements	**Stakeholder Requirement Gathering:** PMs can use LCNC tools to build **interactive mockups** during requirement gathering sessions. Stakeholders can **interact with potential solutions in real-time**, leading to more accurate requirements and reducing scope misunderstandings.
Prepares Business Case	**Project Charter Development:** LCNC platforms can support creating collaborative project charter documents, dashboards, or simple workflows that help define project objectives, success criteria, and high-level scope in an engaging, **visual format** that's easier for stakeholders to understand and approve.
Identifies Risks	**Structured Stakeholder Input Collection:** Instead of traditional note-taking, PMs deploy no-code forms (like Airtable) to **systematically gather stakeholder ideas** and requirements. This creates a structured, sortable database from day one, ensuring no critical input is lost and aiding early risk identification.

3.2 Planning Phase

The Planning Phase involves translating objectives into **detailed tasks and deliverables**. Key activities include defining scope, schedule, resources, and cost baselines, as well as developing detailed quality and risk management plans.[3][4][8]

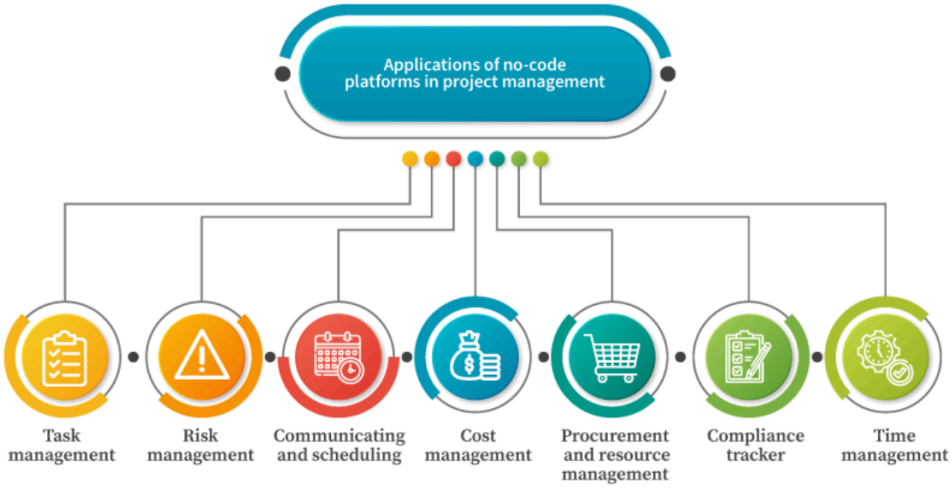


Fig:Applications of No-Code Platforms in Project Management[4]

Traditional Goal	LCNC Enhancement to Project Planning
Defines Scope/Schedule	**Visual Workflow Mapping:** Create **Kanban boards or Gantt charts** using drag-and-drop functionality to map out project dependencies, identify potential bottlenecks, and visualize the entire workflow without coding.
Creates WBS	**Detailed Task Breakdown:** Use drag-and-drop tools to create detailed task breakdowns with clear deadlines, breaking down complex projects into manageable components quickly.
Resource Allocation	**Budget Tracking:** Leverage built-in budget tracking to **allocate resources efficiently** and monitor spending throughout the project lifecycle. **Automated Scheduling** takes advantage of automated features to assign tasks, set reminders, and manage dependencies.

Manages Requirements	**Visual Requirement Management:** Platforms allow requirements, features, and ideas to become **visual cards** that can be organized, prioritized, and categorized collaboratively, directly linking back to the initial PoC (See 3.1).

3.3 Data Analysis Techniques

This phase is where the project plan is **launched and implemented**. It focuses on coordinating teams, managing day-to-day execution, handling issues, and ensuring alignment with objectives.[2][1][8]

Traditional Goal	LCNC Enhancement to Project Execution
Task Management	**Automated Task Management:** Define workflows and automate task creation and assignment based on predefined rules, ensuring everyone understands their responsibilities without manual intervention.
Team Collaboration	**Integrated Collaboration Tools:** Break down communication silos using built-in chat features, document collaboration tools, and discussion forums for **real-time communication** and issue troubleshooting.
Managing Progress	**Real-Time Progress Tracking:** Teams use real-time progress-tracking dashboards where members update tasks, upload deliverables, and collaborate seamlessly—**eliminating the need for endless status update meetings.**
Resource Focus	**Process Automation:** Automate many daily administrative tasks such as resource management, timekeeping, and project progress tracking, allowing project managers to focus on **strategic aspects of work.**

3.4 Monitoring & Controlling Phase

This phase **tracks and measures performance** against baselines using Key Performance Indicators (KPIs) and risk assessments. It involves implementing corrective actions to address deviations and ensure alignment with objectives.[8][6]

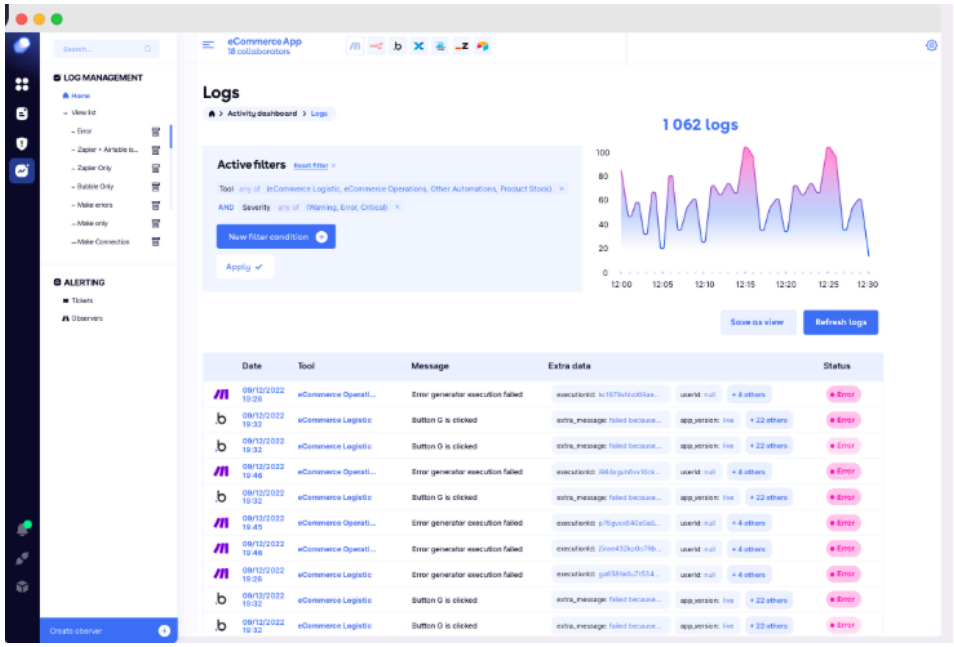


Fig:Monitoring for different projects using Nocode.tech interface[6]

Traditional Goal	LCNC Enhancement to Monitoring & Controlling
Performance Measurement	**Customizable Real-Time Dashboards:** Gain instant insights into project status with dashboards that track key metrics like task completion rates, budget adherence, and resource utilization in a **single, centralized location.**
Proactive Risk Mgmt	**Automated Milestone Alerts:** Set up automated alerts to notify stakeholders of upcoming milestones, missed deadlines, resource overheads, or key project events, ensuring **proactive adjustments** are made.

Issue Handling	**Risk Management:** Document potential risks, assess their impact, and develop mitigation strategies using **built-in risk management tools**, enabling early identification of roadblocks.
Reporting	**Automated Reporting:** Generate real-time reports effortlessly and share project progress with stakeholders, keeping everyone informed and aligned **without manual report creation.**

3.5 Project Closure

The Closure Phase formally **completes project activities**, obtains stakeholder acceptance, finalizes deliverables, and ensures knowledge transfer.[7][8]

Traditional Goal	LCNC Enhancement to Project Closure
Documentation & Handoff	**Centralized Document Management:** Consolidate all project documents, presentations, and final deliverables within the platform, making everything **easily accessible** for future reference and eliminating scattered files.
Lessons Learned	**Built-in Feedback Surveys:** Gather feedback from stakeholders and team members directly within the platform using built-in survey features, simplifying the post-project review process and aiding in the documentation of **lessons learned.**
Archiving	**Automated Archiving:** Leverage archiving capabilities to **securely store project data** and organize it for easy retrieval, fostering knowledge retention and enabling smoother handovers for future projects.

3.6 Bibliography

No-Code Guide for End-to-End Project Management (Full Guide) | Knack[1]

No-Code Tools And Project Management: A Revolution[2]

What is A Project? 2026 Basics for Success | Quixy[3]

Revamp Project Management with No-Code: Solution for Efficient Project Delivery | Quixy[4]

Project Initiation Guide: Start Projects Righ[5]

Why documenting and monitoring are essential in no-code adoption[6]

Project closing[7]

Transforming Enterprise Apps with Low Code/No Code (LCNC) | Webinar[8]

4. Findings and Analysis

While LC/NC platforms are often seen as tools for app creation, modern enterprise use cases show that their true power extends far beyond app development. Today, LC/NC frameworks are enabling Business Process Management (BPM), workflow automation, data visualization, supply chain tracking, finance operations, and internal productivity tools

Gartner (2024) reported that over **65%** of new business process automation projects in enterprises are built using LC/NC solutions rather than traditional development frameworks

4.1 Use Case Analysis

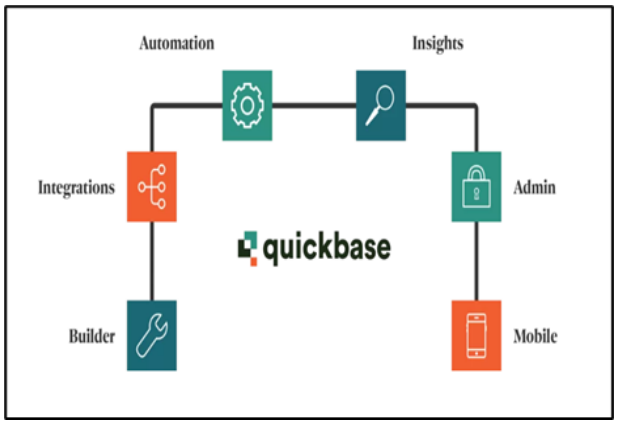
4.1.1 Case Study 1: Schneider Electric – Procurement Workflow Automation (Quickbase)

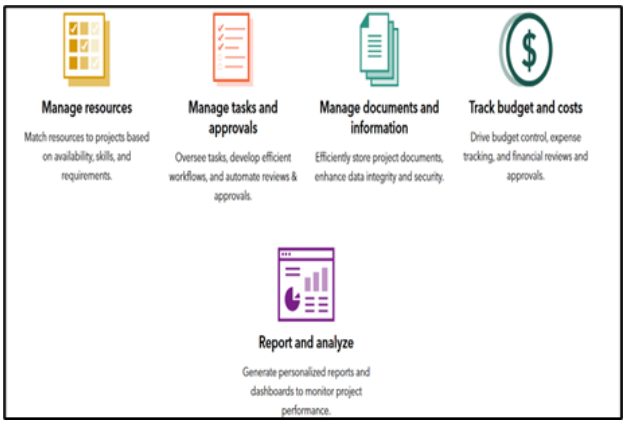
A. **Problem / Issues**
    a. **Fragmented Procurement Systems:** Supplier data, RFQs, and performance reports were scattered across spreadsheets and legacy tools.

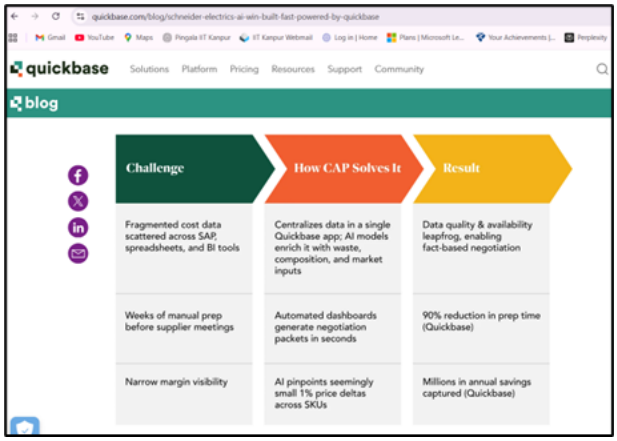
b. **Inefficiency & Duplication:** Employees spent extensive time preparing supplier cost and risk data manually.
c. **Delayed Decision-Making:** Slow information retrieval hindered supplier negotiations and performance evaluations.
d. **Scalability Constraints:** Custom-coded procurement dashboards were expensive and time-consuming to modify.

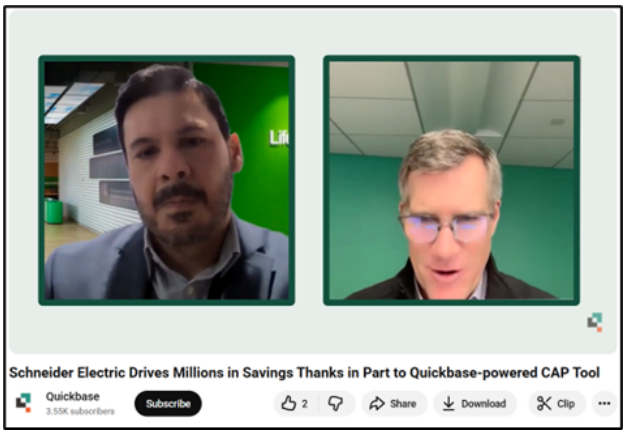
B. No Code Implementation

a. Schneider Electric adopted **Quickbase**, a no-code relational database and workflow automation platform.
b. Created a centralized **Cost Analytics Power (CAP)** tool for Procurement BPM.
c. Business users (non-developers) built automated workflows for supplier scoring, contract tracking, and negotiation readiness.
d. Integrated with ERP and Power BI for real-time analytics**.**









C. Outcome

a. 30 million supplier transactions managed annually through Quickbase.
b. 90% reduction in data preparation time.
c. Millions in annual savings via improved supplier performance tracking.

D. Iron Triangle Impact

Time	Cost	Quality
Reduced supplier prep time by 90%	Lower IT dependency; internal teams maintain workflows	Consistent, accurate supplier data and faster decisions

E. Reference:

a. https://www.quickbase.com/customers/schneider-electric-drives-millions-in-savings-thanks-in-part-to-quickbase-powered-cap-tool
b. https://www.youtube.com/watch?v=rRv_mosJPn
c. https://www.quickbase.com/blog/schneider-electrics-ai-win-built-fast-powered-by-quickbase
d. https://www.thehackettgroup.com/the-hackett-group-announces-winners-of-the-2025-hackett-innovation-awards/?utm_source=chatgpt.com

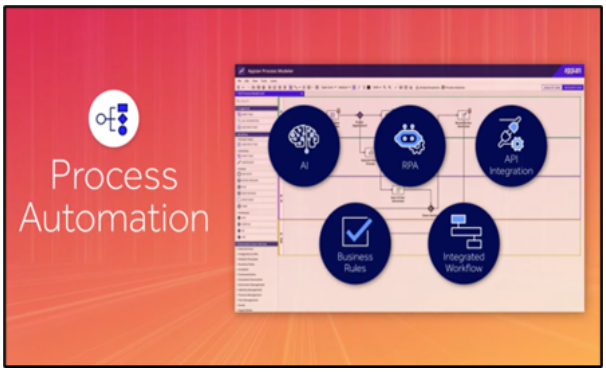
4.1.2 Case Study 2: Pfizer – Clinical BPM Automation (Appian Platform)

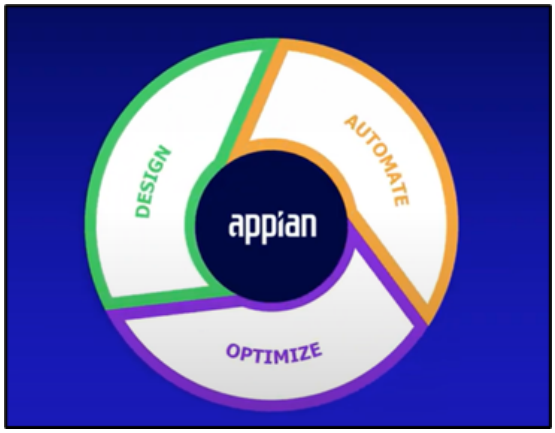
A. Problem / Issues

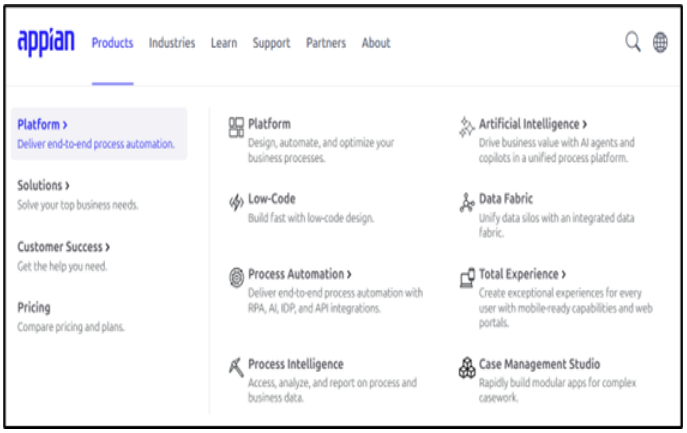
a. **Complex BPM:** Managing regulatory compliance, CRO contracts, and clinical trial submissions required collaboration between R&D, QA, and regulatory teams.
b. **Manual Workflow:** Each phase was tracked through documents and email threads, leading to errors and regulatory delays.
c. **Compliance Risk:** Repetition and lack of audit trails exposed Pfizer to compliance gaps with FDA standards.
d. **Long Approval Cycles:** Paper-based processes delayed market readiness for new drugs.

B. Low Code Implementation

a. Pfizer implemented **Appian**, a **low-code BPM suite** that integrates workflows, documents, and compliance tracking.
b. Created a **Clinical Trial Management System (CTMS)** on Appian to:
    i. Automate approval chains, submission templates, and review processes.
    ii. Connect disparate systems (SAP, document repositories, regulatory databases).
    iii. Use visual modelling to replicate existing business rules.







C. Outcomes

a. Submission turnaround time reduced by 80%.
b. Higher compliance audit scores due to real-time tracking and documentation.
c. Enabled faster coordination between global teams

D. Iron Triangle Impact

Time	Cost	Quality
Major reduction in approval time	Fewer reworks and manual labour costs	Higher accuracy and compliance consistency

E. References

a. https://appian.com/about/explore/customers/all-customers/use-case-clinical-operations?utm_source=chatgpt.com
b. https://www.linkedin.com/posts/70ragtopss_pfizer-brightstartaward-projectunity-activity-7361565026903592960-TqpS?utm_source=share&utm_medium=member_desktop&rcm=ACoAADMQM4IBrES9MDfoljDMY1NBzPoFOgV3bso

c. https://appian.com/

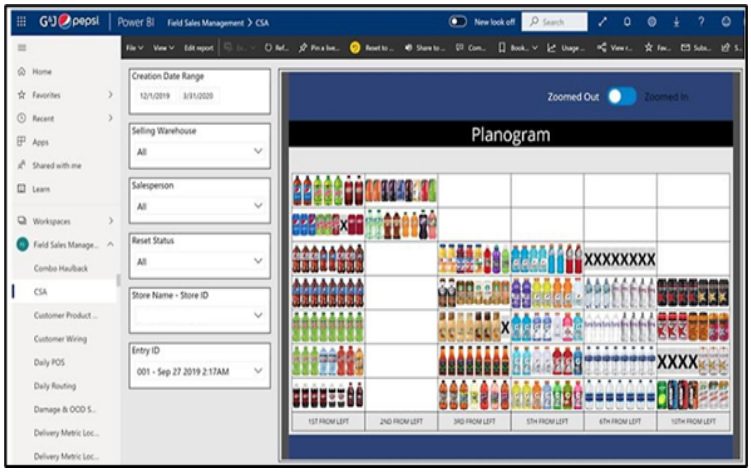
4.1.3 Case Study 3: G&J Pepsi – Field Operations Automation (Microsoft Power Platform)

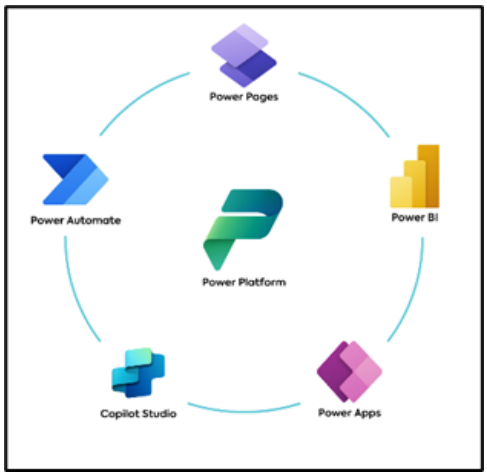
A. Problem / Issues

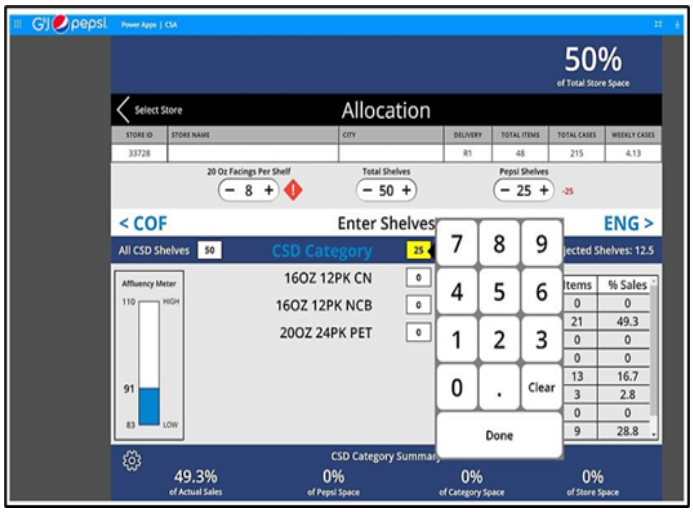
a. **Manual Reporting:** Field representatives entered data on paper or Excel sheets, causing lags in visibility.
b. **Limited IT Support:** The IT team struggled to create mobile-ready tools fast enough for changing field needs.
c. **Inconsistent Data:** Non-standardized reporting formats led to decision delays.
d. **Underutilized Workforce Time:** Managers spent hours compiling field reports manually.

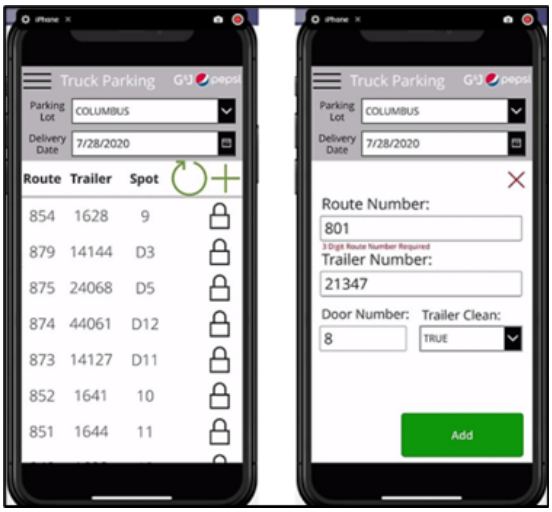
B. Low Code Implementation

a. G&J Pepsi deployed **Microsoft Power Apps**, **Power Automate**, and **Power BI** to digitize its **field workflow** and data capture.
b. Citizen developers in operations built lightweight applications for inventory, sales, and performance reporting.
c. Real-time dashboards enabled visibility for 900+ field employees and managers









C. Outcomes

a. USD 500K saved in the first year, later exceeding USD 1.5M.
b. Faster reporting cycles and reduced administrative burden.
c. Field workforce productivity increased significantly.

D. Iron Triangle Impact

Time	Cost	Quality
Real-time field updates	Reduced admin costs and IT hours	Standardized and accurate reporting

E. References

a. https://www.microsoft.com/en/customers/story/821738-gj-pepsi-cola-bottlers-consumer-goods-microsoft-power-platform?utm_source=chatgpt.com

b. https://rsmus.com/content/dam/rsm/insights/technologies/microsoft/1pdf/gj-pepsi-cola-bottlers-case-study-may2022.pdf?utm_source=chatgpt.com

4.2 Impact on Project Management Constraints

Low-Code/No-Code (LC/NC) platforms are transforming the traditional Iron Triangle of project management—**time, cost, and quality**—by enabling organizations to achieve faster delivery, lower costs, and higher process reliability simultaneously. Schneider Electric's use of **Quickbase** for procurement workflow automation illustrates this shift clearly. Through the creation of its Cost Analytics Power (CAP) tool, the company consolidated supplier data and automated cost and risk analyses, achieving a full ROI in under three months and managing 30 million transactions annually. This automation drastically reduced preparation time while improving data accuracy and decision-making quality. Similarly, **Pfizer's** deployment of the **Appian** platform in clinical trial operations shortened its documentation and approval cycles from 20 days to just 4 days, significantly reducing compliance delays and human errors in regulatory processes. In another instance, **G&J Pepsi's** adoption of the **Microsoft Power Platform** automated over 100 field workflows, empowered 900 field employees, and generated over $1.5 million in cost savings through enhanced visibility and reduced manual reporting. Across these cases, LC/NC platforms have proven to be catalysts for balancing the Iron Triangle—compressing development cycles, cutting operational costs, and improving process quality. Instead of forcing trade-offs between these constraints, LC/NC technology allows organizations to achieve speed, affordability, and accuracy together, redefining how modern projects are executed and managed.

4.3 Addressing the "Is it just for apps?

Contrary to popular belief, **Low-Code/No-Code (LC/NC) platforms are not limited to building apps**; they are powerful enablers of **process automation, workflow optimization, and enterprise-wide digital transformation**. Real-world use cases demonstrate this broader scope. **Schneider Electric's Quickbase implementation** was not an app-development initiative—it was a strategic automation of procurement workflows, integrating supplier performance, cost analytics, and compliance into a unified platform. This initiative streamlined business processes across global operations, eliminating the need for custom application development. Likewise, **Pfizer's Appian-based system** focused on **Business Process Management (BPM)** for clinical trials and regulatory submissions, automating end-to-end documentation and approvals—activities that go far beyond app creation. **G&J Pepsi's Power Platform project** also exemplifies non-app innovation: it

empowered business teams to digitize field operations, reporting, and approval workflows without writing code, transforming everyday processes rather than delivering a standalone product. These examples reveal that LC/NC tools are best understood as process orchestration platforms, enabling citizen developers and project managers to innovate without depending entirely on IT teams. Their value lies not in building apps but in solving business challenges, increasing agility, and bridging operational gaps across departments. Thus, LC/NC platforms represent a paradigm shift—from coding tools to strategic instruments for workflow intelligence and enterprise transformation.

5. Discussion and Recommendations

5.1 Discussion of Findings

This section presents a detailed analysis of how Low-Code and No-Code (LC/NC) platforms are integrated across the Project Management Lifecycle (PMLC).

The section evaluates LC/NC adoption through the lens of PMI's five process groups — Initiation, Planning, Execution, Monitoring & Controlling, and Closure — with a focus on project efficiency, governance, and digital transformation outcomes.

1) Initiation (project idea, business case, stakeholder alignment)

Typical LC/NC uses

- Rapid proof-of-concept (PoC) prototypes / clickable mockups to validate requirements and stakeholder buy-in.
- Quick intake forms to capture project requests and automated routing to PMO.
- Requirement capture templates (no-code forms + shared databases).

Examples

- Microsoft Power Apps + Power Automate: build quick prototype apps or intake forms to capture project requests and route approvals. Microsoft documents patterns for project management prototypes. Microsoft Learn+1
- Airtable / monday.com: quick intake boards or templates to centralize requests and prioritize projects visually. Airtable promotes PM use cases and templates. Airtable+1

Advantages

- Very fast: prototypes and intake forms can be ready in hours–days → better stakeholder engagement and earlier feedback.
- Low cost to validate ideas before heavy investment.
- Democratizes ideation (business users can demonstrate needs).

Disadvantages / limitations

- Early prototypes may not reflect production non-functional needs (performance, security).
- Business users can over-promise features that are hard to scale later.
- Risk of many "approved" prototypes that never get governance (shadow IT).

2) Planning (scope, schedule, resources, risk identification)

Typical LC/NC uses

- Visual project schedules and resource boards (drag & drop Gantt/kanban).
- Risk registers built with forms + automated notifications.
- Estimation calculators (no-code spreadsheets + logic).

Examples

- monday.com: configurable boards with timeline and Gantt views, automations for dependencies and notifications — widely used for PM planning dashboards. monday.com+1

- Airtable: flexible data models to track scope, tasks, resources and risks; AI features to generate setups from plain language. Airtable+1

Advantages

- Faster creation of baseline plans and shared artifacts.
- Easy iteration: change a field and automations update assignments, alerts, dashboards.
- Non-technical PMs can own plan maintenance.

Disadvantages/limitations

- Complex resource optimization and leveling algorithms in enterprise-grade PM tools may be absent or simplistic.
- Integrating schedule with finance/ERP for cost forecasting often requires custom connectors or pro-code.
- Risk of inconsistent data models if multiple teams build their own bases.

3) Execution (tasking, collaboration, automation of workflows)

Typical LC/NC uses

- Automations for task creation, notifications, approvals and handoffs (e.g., when a task completes, create follow-up tasks).
- Lightweight apps for field data capture (mobile forms) and team collaboration.
- Citizen-built microservices / internal tools to replace manual spreadsheets.

Examples

- Zapier / Power Automate: automate cross-app workflows (e.g., when a form is submitted → create tasks in Asana/Jira → notify stakeholders). Zapier documents many PM automation recipes.
- OutSystems / Mendix: used when execution needs more robust internal apps (mobile + offline + integrations) — OutSystems highlights use cases for warehouse/operations apps.

Advantages

- Reduces manual handoffs and email chasing → faster cycle time.
- Field teams can capture data immediately (less rework).
- Empowers subject-matter experts to automate routine processes.

Disadvantages/limitations

- Automations chained across many apps can become fragile if source apps change APIs or fields.
- Quality & maintainability: citizen apps can accrue technical debt (no unit tests, inconsistent patterns).
- For complex business logic or performance needs, low-code may reach its limits.

4) Monitoring & Controlling (status reporting, dashboards, KPIs, audits)

Typical LC/NC uses

- Real-time dashboards with visuals (status, burn-down, risks).
- Automated status-report generation and distribution.
- Event-driven alerts (budget overrun, missed milestones).

Examples

- monday.com dashboards for consolidated PM reporting and visual KPIs. monday.com

- Power BI + Power Apps + Power Automate: create automated reporting pipelines: forms/data → dataset → scheduled report distribution. Microsoft demonstrates project-management patterns that integrate these tools. Microsoft Learn+1

Advantages

- Near real-time visibility (no waiting for weekly status decks).
  Automated, repeatable reporting reduces PM admin time (PPT used this example of large hours savings).
- Dashboards can be tailored per stakeholder.

Disadvantages/limitations

- Data hygiene issues: dashboards are only as good as data quality; inconsistent usage will show misleading KPIs.
- Security & access control: dashboards exposing sensitive info need governance (roles/SSO).
  Scaling: linking dashboards to enterprise data stores (data warehousing) may require pro-code or ETL work.

5) Closure (retrospective, documentation, archival, lessons learned)

Typical LC/NC uses

- Automated collection of lessons learned, project artefacts and client feedback (surveys → summary dashboards).
- App-driven retention & retirement workflows (automate archival to document store).
- Generate closeout reports automatically from project data.

Examples

- Power Automate + Forms to automate collection of post-implementation surveys and compile dashboards. Microsoft
- Airtable can act as a knowledge base for lessons learned and templates for future projects. Airtable

Advantages

- Ensures that closure tasks are not skipped (through automated reminders and archiving).
- A centralised lessons learned repository accelerates organisational learning.

Disadvantages/limitations

- Long-term archival and compliance: storing records in vendor platforms raises retention/export needs — must ensure portability.
- Transforming ad-hoc citizen data into formal audit-quality records may need governance and additional steps.

Detailed Advantage vs. Disadvantage Table:

Advantage (PM perspective)	Concrete benefit	Typical counterpoint / disadvantage
Speed to prototype	Stakeholders can interact with a working PoC in days	PoC drift (requirements grow beyond the original scope)
Lower build cost	Business users build simple apps vs full dev cycles	Hidden TCO: licensing, CoE, integration costs
Democratizes automation	Fewer tickets to IT; PMs automate routine tasks	Shadow IT & security risk without governance
Better visibility	Auto dashboards and alerts reduce manual reporting	Garbage-in → garbage-out (data hygiene problems)
Reuse & templates	Standard templates speed up new project setup	Proliferation of slightly different templates → maintenance overhead

5.2 Views & Recommendation for Implementing LC/NC

Actionable Plan for Safe and Effective LC/NC Adoption

This section outlines a clear, step-by-step plan that project managers or PMOs can follow to adopt **Low-Code and No-Code (LC/NC) platforms** efficiently and securely. It focuses on practical execution, governance, training, and scaling.

A. Strategic Posture — Setting the Foundation

1. **Define your goal:**
   Decide why you want to use LC/NC — whether to speed up project delivery, reduce manual work for PMs, or enable employees to create their own small

tools.
This clarity helps in selecting the right platform and performance metrics.

2. **Adopt a phased approach:**
   Begin with **small pilot projects**, then expand to multiple departments, and finally scale to enterprise level.
   This reduces risk and allows lessons learned from pilots to improve large-scale adoption.
3. **Ensure strong sponsorship:**
   Assign a senior leader or PMO head to oversee the LC/NC initiative. Their support helps secure resources, monitor progress, and align efforts with business goals.

B. Governance and Operating Model — Keeping Control

1. **Set up a Center of Excellence (CoE):**
   A CoE acts as the control center for all LC/NC activities. It defines templates, coding standards, approval processes, training programs, and security rules.
   (As McKinsey & Forrester suggest, this approach transforms scattered "shadow IT" projects into a governed organizational capability.)
2. **Classify applications:**
   Define three app categories — **Citizen apps**, **Departmental apps**, and **Enterprise apps** — and set different approval and review processes for each.
3. **Establish app lifecycle rules:**
   Create guidelines for how apps are created, tested, deployed, updated, backed up, and eventually retired.
4. **Secure data and identity:**
   Enforce **Single Sign-On (SSO)**, restrict unnecessary data access, and ensure compliance with data protection laws.

C. Vendor Selection and Procurement Criteria

When choosing LC/NC tools, focus on the following:

Key Requirement	Description
Integration	Should easily connect with your ERP, HR, and BI systems.
Extensibility	Should allow custom coding if advanced features are needed.
Governance Controls	Must include admin dashboards, audit trails, and user access logs.
Analytics and Monitoring	Should track performance, errors, and app usage.

Vendor Strength	Prefer established platforms with long-term support (e.g., Microsoft Power Platform, OutSystems, Mendix, Airtable, Monday.com).

Validate all vendor costs, including **licenses, training, and maintenance**, before making final decisions.

(MarketsandMarkets and Forrester emphasize considering total cost of ownership.)

D. Pilot Design — Testing Before Scaling

Run **2–3 pilot projects** for 3–6 months to test the practicality of LC/NC solutions.

Pilot Type	Example	Objective
Pilot A (Low complexity)	Automate PM status reporting	Save manual reporting time
Pilot B (Medium complexity)	Automate change request approval workflow	Improve process transparency
Pilot C (High complexity)	Develop an ERP-connected app	Test integration capabilities

KPIs to track:

- Average reporting time per PM (hours/week)
- Cycle time for change requests
- Number of manual emails replaced
- Time from idea → prototype
- Total cost to build (FTE hours, annual cost)

Use real organizational data rather than estimated figures from external studies.

E. Training, Change Management, and Developer Program

1. **Create learning levels:**
   Define roles like *Citizen Developer*, *Advanced Citizen*, and *Pro Developer*, with corresponding training and certifications.
2. **Provide ready-made templates:**
   Offer pre-built modules such as dashboards, risk logs, and timesheets to help beginners start quickly.
3. **Build a community of practice:**
   Organize monthly sessions, show-and-tells, and review boards to share best practices and recognize high-quality apps.

F. Security and Compliance

- Enforce **SSO and conditional access policies** to protect user data.
- Implement **automated security scans** for apps accessing sensitive information.
- Keep an updated **inventory of all active apps**.
- Conduct **quarterly security audits** to identify risks early.
  (McKinsey recommends proactive governance to prevent data breaches and shadow IT.)

G. Integration and Architecture Best Practices

- **Use an API-first approach:** Prefer LC/NC platforms that connect through secure, standard APIs.
- **Centralize data:** Maintain a single master dataset to prevent duplication.
- **Plan for portability:** Ensure that all data and applications can be exported if the vendor changes.

H. Cost Evaluation and ROI Checklist

Include the following when building your LC/NC business case

Cost Element	Description
Licensing Fees	Developer, user, and admin costs.
Integration Effort	Time or consultant cost for linking systems.
Training & CoE Setup	Cost of staff training and resource allocation.
Maintenance & Security	Upgrades, bug fixes, audits.
Expected Savings	Reduction in FTE hours, faster delivery time, fewer third-party costs.

Use data from Forrester's *Total Economic Impact* reports as a reference point for cost–benefit analysis.

I. Scaling and Continuous Improvement

1. After successful pilots, scale by domain — start with HR and Finance, then expand to Operations and Customer Service.
2. Maintain continuous KPI tracking and conduct quarterly reviews to retire unused apps.
3. Invest in advanced tools like **AI-assisted app builders** and **process automation dashboards** as adoption matures.

Time Period	Key Activities
0–3 months	Establish CoE, select pilot cases, baseline KPIs, procure licenses.
3–9 months	Measure pilot success, refine training, roll out to more teams.
9–18 months	Formalize security standards, expand CoE staffing, integrate with enterprise systems.
18+ months	Automate governance, focus PMO on strategic decision-making.

6. Conclusion

The advent of Low-Code/No-Code (LCNC) platforms fundamentally redefines the project management landscape, shifting the Project Manager's role from administrative oversight to **strategic enablement** by accelerating project initiation via rapid prototyping and ensuring efficiency through automated execution. While LCNC promises significant gains in speed and time-to-market, this research critically concludes that these benefits are inseparable from the emerging risks of **Shadow IT, technical debt, and vendor lock-in**. To navigate this new era successfully, organizations must adopt a robust governance framework, specifically by establishing a **Center of Enablement (CoE)**, which is essential for mitigating risks, setting architectural standards, and coaching the next generation of Project Managers who must evolve into data-literate, automation-savvy **Governance Architects**.